

LSP + CLI Live Introspection Integration Plan

Phased implementation directive for configurable editor, CLI, HTTP, and agent-facing compiler observability

Prepared for the Fe origin / trace-ready architecture work - 2026-05-24

Executive stance: do not make the LSP, CLI, or browser UI each invent their own model. Build one shared introspection/query service over canonical compiler trace facts. The CLI, LSP, HTTP endpoints, browser graph views, and agents should all be thin clients over that service.

Decision	Recommendation
First integration milestone	Shared trace/introspection query crate plus JSONL trace artifact boundary. Reports run from facts, not from hard-coded CLI fixtures.
LSP initial feature surface	Keep always-on inlays small: type hints, optional function gas summary, optional loop hotspot chips. Use hover/code lens/commands for richer trace explanations.
Configuration	Typed shared config loaded from defaults + fe.toml + LSP client settings + explicit CLI/env overrides. Expose effective config for humans and agents.
Live CLI-LSP connection	Treat .fe-lsp.json as a versioned endpoint discovery file, not a lock. Add fe lsp status/doctor/stop and fe trace live commands that call typed HTTP/LSP requests.
Gas demo	Start with static bytecode/opcode gas attributed through source maps. Label it conservative/static. Add runtime/test-trace gas later.
Rich IR integration	Worth doing as on-demand views, not as always-on editor noise. Use a common IRView schema with source/origin overlays for AST/HIR/MIR/Sonatina/bytecode/gas.

Audience: implementor agent, compiler/LSP maintainers, and reviewers who need a concrete next push plan.

Review basis: static review of the origin-overhaul-phased branch and the branch files listed in the reference section. This document is architectural guidance, not independent cargo/test verification.

Contents

1. North star and immediate goal
2. Current implementation review
3. Recommended architecture: one introspection spine, many clients
4. Configuration model for LSP, CLI, and live views
5. LSP UX: inlays, hovers, code lenses, commands, and panels
6. CLI UX and live connection to the LSP server
7. Gas-cost demo: minimal but honest
8. Agent-friendly probing UX for Sean's Fibonacci case
9. Forward-looking rich IR integrations
10. Phased implementation plan
11. Acceptance criteria and test plan
12. Risks and anti-patterns
13. Appendices: schemas, endpoints, config, and reference notes

1. North star and immediate goal

The goal is to make the Fe developer experience feel live, explainable, and agent-friendly without recreating the SSOT violations from the earlier instrumentation spike. The LSP, CLI, HTTP/browser views, and agents should all use the same trace facts and the same query/report engine.

The next push should be scaffolding-first: shared introspection service, typed config, trace artifact boundary, live endpoint discovery, and two or three polished LSP/CLI demonstrations. Do not start by adding a large Datalog platform or an always-on graph UI.

1.1 The core product loop

developer edits Fe source

- > LSP observes current workspace/open buffers
- > compiler/introspection emits or refreshes trace facts
- > shared query service derives insights
- > LSP shows small cues inline and rich details on hover/command
- > CLI can ask the same questions in batch or live mode
- > browser/graph view can open the same artifacts for exploration

1.2 The core architectural invariant

The trace/introspection system must remain downstream of compiler-owned truth. HIR, MIR, codegen, backend, and bytecode layers own identities and phase decisions. Trace facts serialize those decisions. Query/report layers derive views. LSP and CLI render those views.

phase-owned compiler facts

- > trace-facts JSONL / in-memory snapshot
- > trace-query / introspection service
- > LSP, CLI, HTTP/browser, agents

This avoids the bad shape:

LSP invents a trace model

CLI invents another report model

HTTP graph view invents node IDs

Datalog schema becomes a second compiler

1.3 What to prove in the next push

1. A shared typed configuration path exists and both CLI and LSP use it.
2. A trace artifact can be emitted, validated, and queried from a downstream report command.
3. The LSP can surface at least one trace-backed hover or inlay using the same query API as the CLI.
4. The live server discovery file is versioned and has status/doctor commands.
5. The gas-cost demo is intentionally modest, labeled static/conservative, and wired through the same architecture.

2. Current implementation review

2.1 What already exists and should be reused

Area	Current implementation	Guidance
LSP server architecture	The language-server crate uses async-lsp, actor-style request handling, debounced multithreaded diagnostics, TCP support, multiple clients, task queue, hover support, and VS Code release infrastructure.	Do not rewrite the LSP. Add shared config and introspection hooks around the existing actor/backend.

Capabilities	Server capabilities already include hover, full text sync, definitions, references, semantic tokens, formatting, inlay hints, code lens, code actions, execute commands, call/type hierarchy, folding, declaration, workspace symbols, and file operations.	There is enough LSP surface. The work is not enabling capability bits; it is wiring useful trace-backed data to the right feature types.
Existing inlays	Inlay hints currently focus on inferred type hints for let statements without explicit annotations.	Keep type hints as the baseline. Add gas/loop/storage inlays behind config gates and thresholds.
Existing hovers	Hover already shows definitions, docstrings, local type/effect information, storage slot/footer details, and documentation navigation context.	Hover is the right place for rich trace explanations, storage history, origin chains, and gas breakdown details.
Codegen views	Existing execute commands include <code>fe.viewMir</code> , <code>fe.viewSonatinaIr</code> , and <code>fe.openDocs</code> ; codegen views are written as virtual files and opened in the client.	Use this pattern for the first IR/introspection panels. Avoid inventing a separate viewer before typed query output exists.
Combined HTTP + WebSocket server	<code>combined_server</code> serves documentation pages and upgrades <code>/lsp</code> WebSocket connections backed by the shared Backend actor.	Extend this server with a small localhost HTTP introspection API. Do not create a separate daemon yet.
Virtual files	<code>VirtualFiles</code> maps generated outputs and builtins into temporary/client-visible URIs.	Use virtual files for MIR/Sonatina/bytecode text views. Use HTTP/browser panels for graph-heavy views.
Logging and diagnosis	Logging already supports <code>FE_LSP_LOG</code> , <code>FE_LSP_CLIENT_LOG</code> , <code>FE_LSP_LOG_DIR</code> , <code>FE_LSP_LOG_FILE</code> , retention, rotation, panic files, and workspace-local defaults.	Keep this. Add trace/query correlation IDs to logs so CLI/live requests can be debugged.
CLI LSP startup	<code>fe lsp</code> supports root selection, TCP mode, combined server mode, writes a <code>.fe-lsp.json</code> server-info file, and handles stale/malformed/root-mismatch cases.	Promote <code>.fe-lsp.json</code> from incidental pointer to versioned endpoint discovery file with status/doctor tooling.
Trace CLI	<code>fe trace</code> currently has <code>loop-cost</code> and <code>explain-local</code> for the Fibonacci fixture.	Keep the UX, but rename/cordon fixture commands and make real reports run from emitted trace facts.
Trace facts	<code>trace-facts</code> defines eight fact variants: origin node, origin edge, compiler event, storage, instruction, instruction category, loop membership, and inline context.	This is the correct fact spine. Extend carefully for gas/opcode only when needed.

2.2 Current gaps to close

Gap	Why it matters	Next action
No central LSP/tooling config model	The LSP README itself lists configurable options and inlay hints as future work. Without a shared config, CLI flags, editor settings, and project config will drift.	Create a typed config crate/module consumed by CLI and language-server.
Trace reports are fixture-backed	The Fibonacci reports are useful UX prototypes, but current facts are constructed in the CLI fixture path, not emitted by compiler phases.	Move toward compiler-emitted trace JSONL and make reports consume a trace snapshot.
No gas functionality yet	A gas hint would be a compelling IDE	Add a conservative static gas demo

	feature, but the current trace command does not include gas data.	from bytecode/opcode/source-map facts.
No live query API	The combined server is docs/LSP-focused; CLI live mode needs typed HTTP or LSP custom requests.	Add a small localhost introspection API behind the combined server.
Server-info file is under-specified	.fe-lsp.json currently has pid, port, workspace_root, docs_url. Good start, but it needs schema/version/capabilities/security fields for reliable CLI discovery.	Define ServerInfo v1 and add fe lsp status/doctor.
Potential duplicate WebSocket paths	combined_server and ws_lsp both handle WebSocket/LSP-ish concerns.	Choose combined_server as the product path; extract shared framing if ws_lsp remains.
Inlays can get noisy quickly	Gas, IR, origin, storage, and loop-cost data can overwhelm the editor.	Default to summary/hotspot hints only. Put detail in hovers, code lenses, commands, and browser panels.

2.3 Current .fe-lsp.json behavior

The CLI already writes a .fe-lsp.json file when running the combined server. The current behavior is closer to “endpoint discovery pointer” than “lock file”: it records process/server information and has stale-file handling. The next push should formalize that role.

Current conceptual role:

.fe-lsp.json ~= best-effort pointer to live LSP/docs server

Recommended role:

.fe-lsp.json = versioned local endpoint discovery file

not configuration

not cache

not compiler state

not authoritative trace data

3. Recommended architecture: one introspection spine, many clients

3.1 Do not make the CLI and LSP talk by scraping text

The tempting shortcut is to make the CLI run LSP commands, or make the LSP shell out to CLI commands and parse text. Avoid that. The durable version is a shared typed query layer.

```
crates/introspection-config    # typed shared config
crates/trace-facts            # emitted facts, JSONL, validation
crates/trace-query            # query engine + typed reports
crates/language-server        # LSP consumer of trace-query
crates/fe                     # CLI consumer of trace-query and live HTTP client
```

3.2 Shared query service shape

Create a small shared service API. This can begin as a Rust trait implemented over an in-memory TraceSnapshot. Later it can be backed by Datafrog, Cozo, or another query engine.

```
pub trait IntrospectionService {
    fn status(&self) -> IntrospectionStatus;
    fn effective_config(&self) -> EffectiveIntrospectionConfig;
    fn loop_cost(&self, request: LoopCostRequest) -> Result<LoopCostReport>;
    fn explain_local(&self, request: ExplainLocalRequest) -> Result<ExplainLocalReport>;
    fn gas_breakdown(&self, request: GasBreakdownRequest) -> Result<GasBreakdownReport>;
```

Fe LSP + CLI live introspection implementation plan - 2026-05-24

```

fn origin_chain(&self, request: OriginChainRequest) -> Result<OriginChainReport>;
fn ir_view(&self, request: IrViewRequest) -> Result<IrView>;
}

```

The LSP, CLI, HTTP server, browser graph view, and agent bridge should call this service or a serialized equivalent. The service must not own compiler identity; it consumes trace facts and current compiler query outputs.

3.3 Data flow

```

compiler/workspace state
-> phase-owned origin/export keys
-> trace-facts emission and validation
-> TraceSnapshot
-> trace-query/introspection reports
-> LSP inlay/hover/code lens/commands
-> CLI batch/live commands
-> HTTP/browser graph endpoints
-> agents

```

3.4 Where Datalog fits later

Datalog is attractive for transitive origin, loop membership, stack-access-for-local, and hoist-candidate queries. But it should arrive after the typed fact stream and reports exist. In the first LSP push, hand-coded query functions over TraceSnapshot are enough.

```

Phase 1: typed facts + hand-coded reports
Phase 2: stable query API
Phase 3: optional Datalog backend behind the same API

```

The API should not leak whether a report was produced by direct Rust traversal or Datalog.

4. Configuration model for LSP, CLI, and live views

4.1 Sources of configuration

Source	Use it for	Examples	Persistence
Defaults	Safe baseline behavior.	Type inlays on; trace/gas/perf hints off or hotspot-only; low analysis budget.	Compiled defaults.
fe.toml project config	Team/project policy and reproducible defaults.	Enable trace JSONL on save; choose gas schedule; cap expensive hints.	Committed with project if desired.
LSP client settings	User/editor preferences and quick toggles.	Turn on gas hints in VS Code/Cursor; hide storage hints; change max origin depth.	Editor/user settings.
CLI flags / env	One-shot dev overrides and debugging.	--trace-level detailed; --live; FE_LSP_LOG; --port.	Ephemeral.
.fe-lsp.json	Live endpoint discovery only.	pid, port, urls, capabilities, token, config hash.	Generated; not user-edited.

4.2 Precedence

```

hard-coded defaults
< fe.toml project config

```

- < LSP initialization options / workspace settings
- < explicit CLI flags / environment debug overrides

The effective configuration should be inspectable from both CLI and LSP. This prevents confusion when a hint is missing because of thresholds, privacy mode, trace mode, or analysis budget.

4.3 Proposed shared config schema

```
[tooling.lsp]
enabled = true
trace_mode = "manual"      # off | manual | on_save | background
max_analysis_ms = 500
max_hints_per_file = 80
```

```
[tooling.lsp.inlay_hints]
types = true
gas = "summary"           # off | summary | hotspots | all
loop_cost = "hotspots"    # off | hotspots | all
storage = "hover-only"    # off | hover-only | hotspots | all
ir_phase = "none"         # none | hir | mir | sonatina | bytecode
```

```
[tooling.lsp.hover]
origin_chain = true
storage_history = true
gas_breakdown = true
max_origin_depth = 8
show_confidence = true
```

```
[tooling.lsp.code_lens]
view_ir = true
trace_loop = true
gas_breakdown = true
open_graph = true
```

```
[tooling.lsp.trace]
emit_jsonl = true
out_dir = "target/fe-traces"
validate = true
level = "summary"         # summary | detailed | exhaustive
```

```
[tooling.lsp.gas]
enabled = true
mode = "static"           # static | test_trace | both
schedule = "cancun"       # explicit EVM revision/schedule
show_uncertain = false
```

```
[tooling.lsp.live]
write_server_info = true
http = true
browser_graphs = true
bind = "127.0.0.1"
require_token = true
```

4.4 LSP client settings shape

Mirror the important settings in LSP client configuration. Avoid exposing every internal field on day one; expose high-level modes.

```
{
  "fe.lsp.inlayHints.types": true,
  "fe.lsp.inlayHints.gas": "summary",
  "fe.lsp.inlayHints.loopCost": "hotspots",
  "fe.lsp.hover.originChain": true,
  "fe.lsp.hover.storageHistory": true,
  "fe.lsp.trace.mode": "onSave",
  "fe.lsp.trace.maxAnalysisMs": 500,
  "fe.lsp.live.browserGraphs": true
}
```

4.5 Config implementation tasks

6. Create a typed FeToolingConfig or IntrospectionConfig struct with serde support.
7. Add loader for defaults + fe.toml + editor settings + CLI overrides.
8. Add LSP workspace/configuration and workspace/didChangeConfiguration handling where the client supports it.
9. Expose fe lsp config --effective and HTTP GET /config/effective.
10. Log effective config hash at LSP startup and trace query time.

5. LSP UX: inlays, hovers, code lenses, commands, and panels

5.1 Rule of thumb

Surface	Best for	Avoid
Inlay hints	Tiny, trustworthy, low-noise cues: inferred types, function gas summary, loop hotspot chip, maybe stack/storage marker when severe.	Long explanations, uncertain origin chains, every opcode/gas detail, per-token spam.
Hover	Rich local context: storage history for local b, gas breakdown for expression/function, origin chain, zext explanation, confidence/provenance.	Large graph traversal, entire function reports, long IR dumps.
Code lens	Actionable entry points: View MIR, View Sonata, View bytecode, Trace loop, Gas breakdown, Open graph.	Frequent per-line noise or non-actionable diagnostics.
Diagnostics	Actual correctness issues or very high-confidence performance smells, preferably opt-in.	Turning every optimization opportunity into a warning.
Commands/panels	Full reports, graph visualization, cross-IR inspection, Webscape Wanderer style exploration.	Information that must be visible for normal editing.

5.2 Initial inlay hint set

Do not start with many hint classes. Start with three, all configurable.

Hint	Example display	Trigger	Default
Type hint	let x: u256	Existing inferred-type logic for unannotated let statements.	On
Gas summary	~1.4k gas static	Function/handler/loop range	Off or summary-only

		has static gas attribution and confidence above threshold.	
Loop hotspot chip	13 inst/iter, 4 zext, 2 stack ops	Loop-cost report identifies a loop with synthetic or storage cost above threshold.	Hotspots only
Storage hint	stack slot	Local has high-confidence stack/memory residency and contributes to loop cost.	Hover-only initially

5.3 Hover content templates

Local hover: storage history

local b

Storage history:

HIR: source local b

MIR: memory_place, reason mutable_local_lowering

Backend: stack slot sp+24, reason frame_slot

Loop impact:

1 stack load / iteration

1 stack store / iteration

Likely compiler area:

scalar promotion / mem2reg / loop-carried SSA

Confidence:

source = compiler-emitted trace facts, validator passed

Loop condition hover: zero-extension explanation

while i < n

Integer normalization in generated code:

i: zero-extended before comparison; i mutates in loop

n: zero-extended inside loop; n is not mutated in loop

Possible compiler opportunities:

preserve known-u32/zero-extension facts for i

hoist or pre-canonicalize n before loop

Data source:

static trace facts + loop membership + compiler events

Function hover: gas summary

recv fib(n: u32)

Static gas estimate:

total: ~X gas under EVM schedule Cancun

top contributors:

loop body: ~Y per iteration, static

storage read/write: ...

call/setup: ...

Caveat:

Static estimate. Runtime gas depends on path, calldata, storage warm/cold state, and loop count.

5.4 Code lenses and commands

Code lens / command	Where shown	Action
View MIR	Function/contract	Existing virtual-file flow.
View Sonatina IR	Function/contract	Existing virtual-file flow.
View bytecode/source map	Function/contract or compiled artifact	Open virtual bytecode view with source-origin annotations.
Trace loop	Loop header	Open loop-cost report panel or virtual text report.
Explain local	Local declaration or use	Open storage/origin explanation.
Gas breakdown	Function/handler/loop	Open static or runtime gas report.
Open origin graph	Any traceable node	Open browser graph view anchored to source/IR/bytecode node.

6. CLI UX and live connection to the LSP server

6.1 CLI should have two modes

Mode	Use case	Data source
Offline / artifact mode	CI, reproducible reports, sharing traces, running from saved build artifacts.	Trace JSONL or compiled artifacts on disk.
Live mode	Use current editor buffers, current LSP workspace state, browser graph, agent probing.	Local LSP/HTTP endpoint discovered via .fe-lsp.json.

6.2 Proposed CLI commands

fe lsp status

fe lsp doctor

fe lsp stop

fe lsp config --effective

fe trace emit <path-or-target> --out target/fe-traces/fib.jsonl

fe trace validate --from target/fe-traces/fib.jsonl

fe trace query loop-cost --from target/fe-traces/fib.jsonl --function main

fe trace query explain-local --from target/fe-traces/fib.jsonl --local b

fe trace query gas --from target/fe-traces/fib.jsonl --range <file:line:col-line:col>

fe trace live loop-cost --uri file:///.../fib.fe --line 12

fe trace live explain-local --uri file:///.../fib.fe --line 9 --local b

fe trace live gas --uri file:///.../fib.fe --range 8:1-16:1

fe trace live open-graph --uri file:///.../fib.fe --line 12

6.3 Server-info file v1

Rename the mental model from “lock file” to “server info / endpoint discovery file.” The file can remain .fe-lsp.json for compatibility, but the schema should be explicit and versioned.

```
{  
  "schema_version": 1,  
  ...  
}
```

```

"pid": 12345,
"started_at_ms": 1779640000000,
"workspace_root": "/path/to/project",
"lsp": {
  "transport": "stdio|tcp|websocket",
  "port": 12345,
  "ws_url": "ws://127.0.0.1:12345/lsp"
},
"http": {
  "base_url": "http://127.0.0.1:12345",
  "docs_url": "http://127.0.0.1:12345/",
  "trace_api_url": "http://127.0.0.1:12345/trace"
},
"capabilities": ["trace.query", "ir.view", "gas.static", "graph.origin"],
"config_hash": "sha256:...",
"auth": {
  "mode": "localhost-token",
  "token_file": ".fe-lsp.token"
}
}

```

6.4 HTTP API endpoints

Endpoint	Purpose	Notes
GET /health	Confirm live server, workspace root, version, capabilities.	Used by fe lsp status.
GET /config/effective	Return effective shared config.	Use for debugging setting precedence.
GET /trace/artifacts	List known trace artifacts and freshness.	Useful for browser and CLI.
POST /trace/query	Run typed query requests: loop-cost, explain-local, gas, origin-chain.	Shared request/response structs with CLI.
GET /ir/view	Return text/graph view for HIR/MIR/Sonatina/bytecode.	Can support virtual files and browser panels.
GET /graph/origin	Return graph JSON anchored to origin key/source range.	For Webscape Wanderer style views.

6.5 Security and reliability

- Bind to 127.0.0.1 by default. Do not expose the introspection API on public interfaces unless explicitly requested.
- Use a random token for HTTP requests that can read workspace state or open files. Store it outside user source if possible.
- Reject paths outside the workspace root unless explicitly allowed.
- Write .fe-lsp.json atomically through a temp file + rename.
- Treat pid liveness as insufficient. Verify /health response, workspace root, and capabilities.
- Handle stale server info by reporting clear status and offering fe lsp doctor cleanup.

7. Gas-cost demo: minimal but honest

7.1 Feasibility

Gas tracing back to source syntax is feasible if it is framed correctly. The minimal version should be static bytecode/opcode gas attribution through bytecode source maps and origin keys. Runtime gas attribution can come later from test traces or EVM execution steps.

7.2 What the first demo should and should not claim

Should claim	Should not claim yet
Static gas estimate for a function, loop, or source range under a named EVM schedule.	Exact user transaction gas under all runtime states.
Opcode/bytecode-cost contributors mapped to source spans where source maps are available.	Perfect attribution through optimizer merges/splits without explicit provenance.
Runtime trace gas for a specific test execution once execution-step facts exist.	Generalized gas for all paths before path-sensitive execution modeling exists.
Confidence/provenance labels: static estimate, runtime trace, posthoc classifier, compiler-emitted.	Unlabeled inlay hints that look authoritative when the estimate is approximate.

7.3 Minimal gas facts

```
pub struct OpcodeFact {
  pub pc: OriginExportKey,    // bytecode pc/range key
  pub opcode: String,
  pub immediate: Option<String>,
  pub category: OpcodeCategory,
}

pub struct GasCostFact {
  pub subject: OriginExportKey, // opcode, pc range, function, or source-origin key
  pub gas_kind: GasKind,       // StaticEstimate | RuntimeTrace
  pub gas: u64,
  pub schedule: EvmSchedule,
  pub confidence: Confidence,
  pub source: GasSource,       // OpcodeTable | EvmTrace | ManualFixture
}

pub struct ExecutionStepFact { // later
  pub trace_id: OriginExportKey,
  pub step_index: u32,
  pub pc: OriginExportKey,
  pub gas_before: u64,
  pub gas_after: u64,
}
```

7.4 Gas in LSP surfaces

Surface	Recommended gas use
Inlay	Only summary numbers: function total static estimate, loop static per-iteration estimate, or hotspot chip. Keep off or summary-only by default.
Hover	Breakdown by source range/opcode category, top contributors, schedule, confidence, static-vs-runtime caveat.
Code lens	Gas breakdown action above functions/contracts/loops.

Command/panel	Full gas attribution tree and graph view: source -> HIR/MIR/codegen -> bytecode pc/opcode -> gas.
CLI	Batch report for CI and regression comparison: diff gas estimates between commits/traces.

7.5 Minimal gas demo milestone

11. Compile or load a small Fe contract fixture with bytecode source map records.
12. Decode bytecode into opcode facts and static gas costs under a named schedule.
13. Join opcode/PC costs to bytecode source-map origin keys.
14. Aggregate to function/range/loop source spans.
15. Expose one inlay and one hover behind config.
16. Make the CLI produce the same report from the same facts.

8. Agent-friendly probing UX for Sean's Fibonacci case

8.1 The agent questions we want to support

Question	Required facts/query support	Best UX
Why is b on the stack?	Storage facts by phase; origin links from loads/stores to local b; loop membership.	Hover on b; CLI explain-local; graph view anchored to b.
Where are the zero-extends coming from?	Instruction category zero_extend; compiler events for integer legalization; origin link to comparison operands.	Hover on while condition; CLI zext report; loop-cost report.
Is n being normalized inside the loop despite being invariant?	Loop membership; def/use/mutation or value property facts; zero-extend events.	Hover on n or while condition; code lens Trace loop.
Did inlining make this worse?	Inline context facts; loop cost per instance; storage facts per runtime/inline instance.	CLI compare; graph view; code lens on callsite.
Which compiler area should Sean investigate?	Report-level heuristics over storage/event/value-property facts.	Report summary with confidence and evidence keys.

8.2 Minimal query responses

```

LoopCostReport {
  loop_key,
  source_range,
  rows: Vec<InstructionAttribution>,
  summary: { total_inst, zext, stack_loads, stack_stores, moves, branches },
  findings: Vec<Insight>,
  confidence,
  input_trace_hash,
}

```

```

ExplainLocalReport {
  local_key,
  display_name,
  storage_history: Vec<StorageStep>,
  loop_impacts: Vec<LoopImpact>,
  origin_chains: Vec<OriginChain>,
  likely_compiler_areas: Vec<Suggestion>,
  confidence,
}

```

```

}

ZextReport {
  source_range,
  operands: Vec<OperandNormalization>,
  hoist_candidates: Vec<HoistCandidate>,
  missing_value_facts: Vec<MissingFact>,
}

```

8.3 Make results agent-friendly

- Return structured JSON with stable origin keys, source ranges, confidence, and evidence records, not just prose.
- Include suggested next probes, e.g. “compare inlined vs non-inlined instance” or “show storage facts for b”.
- Include data-source labels: fixture, compiler-emitted, static estimate, runtime trace, posthoc classifier, backend reason.
- Keep each operation small and typed. Agents should not need to scrape LSP markdown hovers.

9. Forward-looking rich IR integrations

9.1 What “rich IR integration” means

A rich integration should let a user or agent move between source, HIR, MIR/runtime, Sonatina pre/post opt, bytecode/EVM opcodes, gas, execution traces, and graph relationships without losing identity.

```

source span
  <-> HIR node
  <-> MIR/runtime statement/local/value
  <-> Sonatina instruction/value/block
  <-> bytecode PC/range/opcode
  <-> gas cost / execution step
  <-> source map / debug view / graph node

```

9.2 Common IRView schema

```

pub struct IrView {
  pub view_id: String,
  pub kind: IrKind,           // Ast, Hir, Mir, SonatinaPreOpt, SonatinaPostOpt, Bytecode, Gas, ExecutionTrace
  pub owner: OriginExportKey,
  pub source_uri: Option<String>,
  pub nodes: Vec<IrNode>,
  pub edges: Vec<IrEdge>,
  pub text: Option<String>,
  pub overlays: Vec<IrOverlay>,
  pub generated_at: TraceGeneration,
}

pub struct IrNode {
  pub key: OriginExportKey,
  pub label: String,
  pub range: Option<SourceRange>,
  pub phase: CompilerPhase,
}

```

```
pub struct IrOverlay {
  pub node: OriginExportKey,
  pub kind: OverlayKind,    // Origin, Gas, Storage, Hash, Diagnostic, Cost
  pub display: String,
  pub confidence: Confidence,
}
```

9.3 Is it worth it?

Reason it is worth it	Condition
Compiler developers can quickly identify where structure was lost across phases.	IR views are on-demand and anchored to origin keys.
Agents get stable operations for explanation and debugging.	Responses are typed and evidence-backed.
Users can inspect gas/source/bytecode attribution without reading raw compiler internals.	LSP default stays low-noise; detail lives in hovers/panels.
Graph visualization can become a real differentiator.	Graph is a view over trace facts, not a separate graph truth.

It is probably not worth it if the team tries to make every IR visible all the time in the editor, or if graph views are built before trace facts and validation are solid. Start on-demand, scoped to one source range or one function.

9.4 Webscape Wanderer style browser integration

The browser view should be an exploratory graph and artifact workspace, not a replacement for editor hints. It can show nodes, phases, source ranges, storage/gas overlays, and path witnesses from source to bytecode/opcode. The LSP and CLI should open it to a specific anchor.

```
fe trace live open-graph --uri file:///.../fib.fe --line 12
-> reads .fe-lsp.json
-> POST /trace/query { kind: "origin_graph", anchor: source range }
-> opens http://127.0.0.1:<port>/graph?anchor=...
-> graph requests IRView/Graph JSON from same introspection API
```

10. Phased implementation plan

Phase 0 - Stabilize scope and naming

Task	Acceptance criteria
Rename/cordon fixture commands.	Fibonacci fixture UX remains available, but commands and output say fixture/demo unless facts are compiler-emitted.
Document .fe-lsp.json as server-info/discovery.	No code or docs call it compiler state or config. Schema proposal is checked in.
Create current-state inventory.	List LSP features, CLI commands, config inputs, trace facts, and known gaps.

Phase 1 - Shared config and effective-config plumbing

Task	Acceptance criteria
Create shared config structs.	language-server and fe CLI both depend on the same config type.
Load fe.toml project config.	Missing config uses defaults. Invalid config has clear diagnostics.
Handle LSP client settings.	didChangeConfiguration refreshes inlays/hovers without restart where possible.
Expose effective config.	fe lsp config --effective and GET /config/effective show the

	same values.
--	--------------

Phase 2 - Trace artifact boundary

Task	Acceptance criteria
Add trace emit/load/validate CLI.	fe trace emit, fe trace validate, and fe trace query can operate on JSONL.
Reports consume TraceSnapshot.	loop-cost/explain-local no longer require build_fib_trace hard-coding for their report logic.
Add trace source labels.	Reports say fixture/compiler-emitted/static/runtime/posthoc/backend-reason as appropriate.

Phase 3 - Introspection query crate

Task	Acceptance criteria
Introduce trace-query/introspection API.	CLI and LSP can call the same Rust functions for loop-cost/explain-local/gas skeleton.
Define typed report structs.	Reports have source ranges, origin keys, confidence, evidence, and suggested next probes.
Keep Datalog optional/deferred.	No public API depends on a specific query engine.

Phase 4 - Minimal LSP feature integration

Task	Acceptance criteria
Add config-gated gas/loop inlays.	Inlays are off/summary/hotspot by config and respect max_hints_per_file.
Add trace-backed hovers.	Hover on local/loop/function can call shared query service and show confidence/provenance.
Add code lenses/commands.	Trace loop, explain local, gas breakdown, open graph are available where facts exist.

Phase 5 - Live CLI-to-LSP connection

Task	Acceptance criteria
Version .fe-lsp.json.	Includes schema_version, pid, URLs, workspace_root, capabilities, config_hash, auth metadata.
Add status/doctor.	fe lsp status confirms live server; doctor explains stale/root mismatch/malformed/token errors.
Add live trace commands.	fe trace live loop-cost/explain-local/gas call the live server and fallback gracefully.

Phase 6 - Gas demo

Task	Acceptance criteria
Decode bytecode/opcode facts.	Trace snapshot includes opcode facts tied to bytecode PC/range origin keys.
Add static gas facts.	Uses named EVM schedule and labels confidence/static.
Surface in LSP/CLI.	Function gas inlay + hover breakdown + CLI gas report use same facts.

Phase 7 - Rich IR and graph views

Task	Acceptance criteria
Define IRView schema.	AST/HIR/MIR/Sonatina/bytecode views can share

	nodes/edges/overlays structure.
Add HTTP graph endpoints.	Browser view opens anchored graphs from LSP/CLI commands.
Add first cross-IR path witness.	Source range to bytecode/gas path can be displayed and exported.

11. Acceptance criteria and test plan

11.1 Unit and integration tests

- Config precedence tests: defaults < fe.toml < LSP settings < CLI/env overrides.
- Server-info tests: stale file, malformed file, root mismatch, live health success, token mismatch.
- Trace validation tests: missing origin node, duplicate instruction, category without instruction, loop membership without instruction, invalid strings.
- Report recomputation tests: summary counts match instruction rows and categories.
- LSP inlay tests: enabled/disabled by config, max hints respected, no blocking on missing trace.
- Hover tests: local b storage history, loop zext explanation, function gas summary when facts exist.
- CLI parity tests: CLI report and LSP hover/code lens use same query/report structs.
- Gas demo tests: named schedule included, static-vs-runtime caveat included, source attribution is absent/unknown where mapping fails.

11.2 Manual demo checklist

17. Start fe lsp in a workspace and verify .fe-lsp.json is written with schema_version and /health works.
18. Open a Fe file in VS Code/Cursor and verify type hints still work.
19. Enable loop-cost hints and verify only hotspot loops receive hints.
20. Hover over the Fibonacci local b and see storage history from a trace snapshot.
21. Run fe trace live explain-local for the same local and verify the same evidence appears.
22. Enable gas summary mode and verify function-level static gas inlay plus hover caveat.
23. Run fe lsp config --effective and compare with GET /config/effective.
24. Open a graph view anchored to a loop or local and verify source/IR/bytecode nodes share origin keys.

12. Risks and anti-patterns

Risk	Why it hurts	Prevention
LSP becomes the fact owner	Editor behavior diverges from CLI and build artifacts.	LSP only consumes trace-query/introspection service.
CLI scrapes hover markdown	Agent tooling becomes brittle and loses structured evidence.	Expose typed JSON query responses.
Gas hints overclaim precision	Developers lose trust quickly.	Always label static vs runtime, schedule, confidence, and unknown mappings.
Inlay spam	Users turn the feature off.	Default to summary/hotspot; expose thresholds and max hints.
Config sprawl	Same setting has different names/semantics in CLI/LSP/fe.toml.	Single shared config struct and effective-config output.
Graph UI becomes a second truth	Recreates SSOT violations visually.	Graph JSON is generated from trace facts/query reports only.
Live server security assumptions	Local HTTP endpoints can leak workspace data.	Bind localhost, token gate, workspace-root path restrictions.
Datalog too early	A query platform can outrun the fact model.	Implement hand-coded reports first, keep query backend replaceable.

13. Appendices

Appendix A - Custom LSP request/command names

Custom requests:

```
fe/introspectionStatus
fe/effectiveConfig
fe/traceQuery
fe/irView
fe/gasBreakdown
fe/originGraph
```

Execute commands:

```
fe.viewMir      # existing
fe.viewSonatinaIr  # existing
fe.openDocs     # existing
fe.traceLoop
fe.explainLocal
fe.gasBreakdown
fe.openOriginGraph
fe.emitTraceArtifact
```

Appendix B - Trace query request sketch

```
#[serde(tag = "kind", rename_all = "snake_case")]
pub enum TraceQueryRequest {
    LoopCost { uri: String, position: Option<Position>, function: Option<String> },
    ExplainLocal { uri: String, position: Option<Position>, local: Option<String> },
    GasBreakdown { uri: String, range: Option<Range>, mode: GasMode },
    OriginChain { anchor: OriginAnchor, max_depth: usize },
    IrView { uri: String, range: Option<Range>, phase: IrKind },
    OriginGraph { anchor: OriginAnchor, budget: GraphBudget },
}
```

Appendix C - Insight schema sketch

```
pub struct Insight {
    pub title: String,
    pub severity: InsightSeverity,    // Info | Hint | Warning
    pub range: Option<SourceRange>,
    pub confidence: Confidence,       // Exact | High | Medium | Low | Unknown
    pub data_source: DataSource,      // CompilerTrace | Fixture | StaticEstimate | RuntimeTrace | PosthocClassifier
    pub evidence: Vec<EvidenceRef>,   // origin keys, fact ids, trace hash
    pub summary: String,
    pub details: Vec<String>,
    pub suggested_actions: Vec<SuggestedAction>,
}
```

Appendix D - Minimal gas hint policy

Mode	Behavior
off	No gas inlays or hovers unless a command explicitly requests gas.
summary	Function/handler-level gas summary inlay only. Hover

	shows breakdown.
hotspots	Function summary plus loop/range hints only when above threshold.
all	Detailed gas hints across mapped ranges. Developer-only; may be noisy.

Appendix E - Evidence reviewed

language-server README: crates/language-server/README.md - async-lsp actor architecture, diagnostics, TCP/multi-client, hover, TODOs for configurable options and inlays.

LSP capabilities: crates/language-server/src/capabilities.rs - hover, sync, symbols, inlay, code lens, code actions, execute commands, file operations.

LSP handlers: crates/language-server/src/handlers.rs - initialization, workspace discovery, file watchers, open/change/save, diagnostics, hover.

LSP backend: crates/language-server/src/backend/mod.rs and db.rs - Backend state, worker runtime, virtual file mapping, LS DB traits.

Inlay hints: crates/language-server/src/functionality/inlay_hints.rs - current inferred-type hint behavior.

Hover: crates/language-server/src/functionality/hover.rs - definitions, docstrings, types/effects, storage/footer details.

Codegen view: crates/language-server/src/functionality/codegen_view.rs - fe.viewMir, fe.viewSonatinaIr, fe.openDocs virtual-file flow.

Combined server: crates/language-server/src/combined_server.rs - HTTP docs + /lsp WebSocket over shared Backend actor.

Virtual files: crates/language-server/src/virtual_files.rs - generated output and builtin URI mapping.

Logging: crates/language-server/src/logging.rs - FE_LSP_LOG*, log directory precedence, retention/rotation, panic files.

CLI main: crates/fe/src/main.rs - lsp command, root/port/tcp mode, .fe-lsp.json server info handling, trace command enum.

Trace CLI fixture: crates/fe/src/trace.rs - fib fixture trace, loop-cost/explain-local reports, hard-coded facts, no gas command yet.

trace-facts: crates/trace-facts/src/fact.rs and validate.rs - eight fact variants and validator baseline.

Appendix F - Final directive to implementor agent

Build the live CLI/LSP integration as shared infrastructure over trace facts. First make config, trace artifacts, query reports, and server discovery reliable. Then add small configurable LSP hints/hovers. Treat gas as a conservative static demo until runtime gas trace facts exist. Treat rich IR/browser views as on-demand projections, not always-on editor decorations.